

Build a Scratch ORM with ADO.NET

Implementing an Entity Framework–style data mapper and migration CLI on top of PostgreSQL

ORM ImplementationCommand-based MigrationsPostgreSQLADO.NET / NpgsqlC# ReflectionGitHubSubmission

Overview

Most .NET developers use Entity Framework Core without understanding what happens beneath it. In this assignment you will build a simplified ORM from scratch using ADO.NET and Npgsql — the low-level building blocks that production ORMs sit on top of.

You will implement a library that maps C# classes to PostgreSQL tables, supports CRUD through a generic `DbSet<T>`, ships a command-line migration tool, and includes a step-by-step coding demo that proves the whole system works against a live database exactly as a real consumer would use it.

Learning Objectives

Understand how ORMs work internally by building one from the ground up

Use ADO.NET with Npgsql (NpgsqlConnection, NpgsqlCommand, NpgsqlDataReader) for raw PostgreSQL access

Apply C# reflection to map class properties to columns dynamically

Map both value types and nullable instance types to their Postgres-native equivalents

Design a command-driven migration CLI that tracks and applies schema changes

Demonstrate the system through a realistic step-by-step coding walkthrough

Project Structure

Create a .NET solution with two console projects:

MiniOrm/ (Console Application) |─ Attributes/ |─ TableAttribute.cs |─ ColumnAttribute.cs |─
PrimaryKeyAttribute.cs |─ Models/ |─ Product.cs |─ Order.cs |─ Data/ |─ DbContext.cs |─ DbSet.cs |
|─ TypeMapper.cs |─ EntityMetadata.cs |─ Program.cs (demo walkthrough) MiniOrm.Migrations/ (Console
Application) |─ Commands/ |─ MigrationRunner.cs |─ Program.cs (CLI entry point)

Tasks – click to expand

01 Project setup & PostgreSQL connection 15 pts

In the MiniOrm console project, add Npgsql as the only permitted NuGet dependency. Create a DbContext base class in Data/DbContext.cs that accepts a Postgres connection string and manages the NpgsqlConnection lifecycle. Concrete context

classes should inherit from it and declare their DbSet<T> properties.

```
public class AppDbContext : DbContext {  
  
    public DbSet<Product> Products { get; set; }  
  
    public DbSet<Order> Orders { get; set; }  
  
    public AppDbContext(string connStr) : base(connStr) { }  
  
}
```

Only Npgsql is permitted. No Dapper, EF Core, or any other data-access library.

02 Entity mapping & type resolution via reflection 20 pts

Define custom attributes in Attributes/ — [Table], [Column], [PrimaryKey] — and implement a TypeMapper in Data/TypeMapper.cs that uses C# reflection to build runtime metadata for each entity. **Skip any property without a [Column] or [PrimaryKey] attribute** — this prevents navigation properties from being mapped.

C# type	Postgres type	Nullability
Value types – non-nullable		
int (PrimaryKey)	SERIAL	PRIMARY KEY
int	INTEGER	NOT NULL
long	BIGINT	NOT NULL
float	REAL	NOT NULL
double	DOUBLE PRECISION	NOT NULL
decimal	NUMERIC	NOT NULL
bool	BOOLEAN	NOT NULL
DateTime	TIMESTAMP	NOT NULL
Guid	UUID	NOT NULL
Nullable instance types – T?		
int?	INTEGER	NULL
long?	BIGINT	NULL
float?	REAL	NULL
double?	DOUBLE PRECISION	NULL
decimal?	NUMERIC	NULL
bool?	BOOLEAN	NULL
DateTime?	TIMESTAMP	NULL
Guid?	UUID	NULL
Reference types		
string	TEXT	NOT NULL
string?	TEXT	NULL

Use Nullable.GetUnderlyingType(type) to detect nullable value types at runtime. Properties without [Column] attributes are ignored — they do not generate columns.

03 Generic DbSet<T> with CRUD operations 25 pts

Implement a generic DbSet<T> class in Data/DbSet.cs that uses the reflection metadata from Task 02 to dynamically build

and execute parameterised SQL. No table name or column name may be hardcoded. Null values from nullable properties must be passed as DBNull.Value in Npgsql parameters.

Method	Behaviour
Insert(T entity)	Parameterised INSERT, returns new row ID via RETURNING id
FindById(int id)	SELECT WHERE on PK, maps result to T — null columns → nullable property
GetAll()	SELECT * mapped to IEnumerable<T>
Update(T entity)	SET clause from all non-PK properties, WHERE on PK
Delete(int id)	DELETE WHERE on PK

Use parameterised queries with NpgsqlParameter for all values — never concatenate values into SQL strings. Null values must round-trip correctly.

No LINQ provider, lazy loading, or relationship mapping required.

04 Command-based migration CLI 25 pts

In the MiniOrm.Migrations console project, implement a CLI that reads entity metadata from the MiniOrm project, diffs the current database schema against registered entities, and generates or applies DDL accordingly. All generated SQL must use Postgres-native types from the mapping in Task 02, respecting nullability.

Command	Pts	Behaviour
dotnet run -- migrations add <Name>	8	Generates a timestamped .sql file with -- up and -- down sections. Does not touch the database.
dotnet run -- migrations apply	7	Runs all pending migrations in order. Records each in a __migrations table.
dotnet run -- migrations list	5	Prints all files marked [applied] or [pending].
dotnet run -- migrations rollback	5	Reverts the last applied migration using its -- down script.

-- up

```
CREATE TABLE IF NOT EXISTS products (  
  id SERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  price NUMERIC NOT NULL,  
  discount NUMERIC NULL,  
  in_stock BOOLEAN NOT NULL  
);
```

-- down

```
DROP TABLE IF EXISTS products;
```

Scope: CREATE TABLE and ADD COLUMN in up, DROP TABLE and DROP COLUMN in down. Column nullability must match the C# property type.

05 Step-by-step coding demo in Program.cs 15 pts»

In MiniOrm/Program.cs, implement a demo that walks through the exact coding sequence a real developer follows when using your ORM. Define entities, create a context, register DbSet, call all CRUD operations, and print output at each step.

Step 1 Define entities with attributes

```
[Table("products")]

public class Product {

    [PrimaryKey] public int Id { get; set; }

    [Column("name")] public string Name { get; set; }

    [Column("price")] public decimal Price { get; set; }

    [Column("discount")] public decimal? Discount { get; set; } // nullable

    [Column("in_stock")] public bool InStock { get; set; }

}
```

Step 2 Create the DbContext and register DbSet

```
public class AppDbContext : DbContext {

    public DbSet<Product> Products { get; set; }

    public AppDbContext(string connStr) : base(connStr) { }

}
```

Step 3 Instantiate DbContext and run migrations

```
var connStr = Environment.GetEnvironmentVariable("MINIORM_CONN");

var db = new AppDbContext(connStr);

// Before using DbSet, from MiniOrm.Migrations run:

// $ dotnet run -- migrations apply
```

Step 4 Insert entities via DbSet

```
var keyboard = new Product { Name = "Keyboard", Price = 89.99m, Discount = null, InStock = true };

int id = db.Products.Insert(keyboard);

// Output: Inserted Product Id=1, Discount=NULL ✓
```

Step 5 Query, update, delete – verify final state

```
var found = db.Products.FindById(1);

// Output: Found → Keyboard, Price=89.99, Discount=NULL

found.Price = 79.99m; found.Discount = 5.00m;

db.Products.Update(found);
```

```
// Output: Updated → Price=79.99, Discount=5.00 ✓  
  
db.Products.GetAll(); db.Products.Delete(1);  
  
// Output: Deleted Id=1 ✓ — 0 products remaining
```

Read the connection string from the environment variable `MINIORM_CONN` — never hardcode credentials. Document setup in your [README](#).

The demo is a working proof that a real consumer can instantiate a context, register `DbSets`, run migrations, and perform every operation end-to-end against a live Postgres database.

Grading Rubric

15 pts

`DbContext` base class with `Npgsql` connection and `DbSet` registration

20 pts

Reflection-based mapping with full nullable type resolution and attribute-only filtering

25 pts

Generic `DbSet<T>` with all 5 parameterised CRUD operations, null-safe

25 pts

Migration CLI — add (8), apply (7), list (5), rollback (5)

15 pts

Demo in `Program.cs` covers all 5 steps end-to-end, exercises nullable fields, clear console output

Total 100 pts

Submission Requirements

Format Public GitHub repository link submitted via the course portal

Repo structure Two console projects in one solution: `MiniOrm/` (library + demo) and `MiniOrm.Migrations/` (CLI)

README Postgres setup, how to set `MINIORM_CONN`, how to run migrations, how to run the demo, explanation of type mapping and attribute filtering

Restriction Only `Npgsql` permitted as third-party NuGet. EF Core, Dapper, or any other ORM = zero marks for affected task

Deadline 13 May 2026